ENTERPRISE APPLICATION DEVELOPMENT

AURA VIRTUAL TRY-ON

System Documentation & Technical Specification

TECHNOLOGY STACK
PYTHON + POSTGRESQL

ACADEMIC TERM
SEMESTER 4 | 2026

PREPARED BY
Saamyak & Development Team

SUBMITTED TO
EAD Instructor
Enterprise Application Development / DBMS

DOCUMENT MAP

01 OVERVIEW

02 PROJECT PLAN

03 UML DIAGRAMS

04 ARCHITECTURE

05 DATABASE

06 USER MANUAL

07 INSTALLATION

Document Information

PROJECT	TECHNOLOGY
Aura SaaS	Python, PostgreSQL
VERSION	DATE
1.0 (Final)	2026

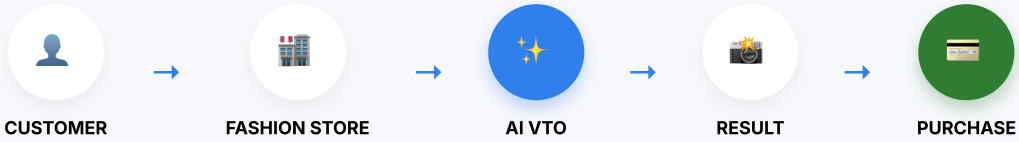
TABLE OF CONTENTS

1. Overview of the Application 03
2. Project Plan 06
3. UML Diagrams 09
4. System Architecture 12
5. Database Design 14
6. User Manual 17
7. Application Installation Manual 19

01

OVERVIEW

AURA VIRTUAL TRY-ON



ARCHITECTURE	CORE FEATURE	PLATFORM	TECH STACK
MULTI-TENANT	AI TRY-ON	E-COMMERCE	PYTHON+PGSQL

1.1 Introduction

Aura is an advanced, highly scalable Software-as-a-Service (SaaS) platform built for the e-commerce fashion industry. It empowers independent fashion brands and retail enterprises to launch fully customized digital storefronts equipped with an AI-powered Virtual Try-On (VTO) engine.

01

FOUNDATION

1.2 Problem Statement

Traditional e-commerce fashion suffers from exceptionally high return rates (exceeding 30%). This is heavily driven by sizing ambiguity and customers' inability to accurately visualize garments on their specific body shapes using standard model photography.

1.3 Proposed Solution

Aura provides a "Fit Passport." Customers upload a photo, and using state-of-the-art Generative AI diffusion models, the platform realistically renders clothing onto the customer's photo, preserving fabric draping, textures, and lighting.

1.4 Purpose of the Application

The system is designed to reduce clothing return rates, increase conversion rates, and provide an enterprise-grade backend where thousands of brands can host their custom domains from a single unified codebase.

TARGET USERS

Platform Admin

Manages the global SaaS infrastructure, configures AI engines, monitors system health, and manages billing for tenant brands.

Brand Owner

Acts as the tenant. Customizes their storefront theme, maps custom domains, manages product catalogs, and fulfills e-commerce orders.

Customer

The end-user shopper who creates a Fit Passport, utilizes the AI Virtual Try-On functionality, and completes purchases.

KEY OBJECTIVES

1. **Scalable Architecture:** Deliver a Python/Django based web application with seamless PostgreSQL tenant isolation.
2. **Real-Time UI:** Implement WebSocket communication via Daphne to stream asynchronous AI progress without blocking.
3. **AI Integration:** Abstract GPU tasks via Celery and Redis to interface cleanly with IDM-VTON and Fashn models.
4. **E-Commerce Suite:** Provide a complete shopping cart, inventory management, and checkout pipeline for every tenant.

01

FEATURES & REQUIREMENTS

✨ AI VIRTUAL TRY-ON

Utilizes diffusion models to render clothing onto customer photos realistically. Handles pose adaptation and fabric draping.

GENERATIVE AI

🏢 MULTI-TENANT ISOLATION

A single deployment handles thousands of brands. Each brand receives an isolated dashboard and catalog via strict ORM filtering.

POSTGRESQL ROW SECURITY

🌐 CUSTOM DOMAINS

Brands automatically map subdomains or external domains. A custom Django middleware intercepts Host headers to route traffic.

MIDDLEWARE ROUTING

🛒 E-COMMERCE CORE

Complete shopping workflow including inventory tracking, variants, cart sessions, order status, and checkout processing.

TRANSACTIONAL LOGIC

FUNCTIONAL REQUIREMENTS

ID	REQUIREMENT	DESCRIPTION	PRIORITY
FR-01	Tenant Data Isolation	Brands must exclusively access their own products, orders, and configurations.	HIGH
FR-02	Domain Resolution	System must resolve brands based on HTTP Host headers dynamically.	HIGH
FR-03	VTO Image Generation	Users can upload photos to trigger background GPU image generation tasks.	HIGH
FR-04	Checkout Workflow	Users can aggregate items into a cart session and execute standard checkout.	MEDIUM
FR-05	Catalog Management	Brands can perform CRUD operations on their specific inventory items.	MEDIUM

NON-FUNCTIONAL REQUIREMENTS

ID	REQUIREMENT	DESCRIPTION	PRIORITY
NFR-01	Asynchronous Flow	AI generation must execute in background workers to prevent thread blocking.	HIGH
NFR-02	Real-Time Updates	Progress updates must stream to the client interface via WebSockets (WSS).	HIGH
NFR-03	Database Scalability	The schema and indices must support high concurrency across thousands of tenants.	MEDIUM

02

PROJECT PLAN

DEVELOPMENT LIFECYCLE

PLANNING — ANALYSIS — DATABASE — DEVELOPMENT — AI INT. — DEPLOYMENT

1. Planning & Analysis

Requirements gathering, validating Generative AI models (IDM-VTON, Fashn), and defining strict multi-tenant constraints for SaaS architecture.

2. Architecture & DB

Establishing the Django backend, PostgreSQL schema, and the CustomDomainMiddleware for dynamic Host-based tenant routing.

3. E-Commerce Core

Developing the shopping mechanisms, cart sessions, inventory models, and order lifecycle management within the isolated tenant contexts.

4. AI / ASGI Integration

Connecting the Python backend to asynchronous Celery GPU queues via Redis, and establishing Daphne WebSockets for real-time streaming.

5. UI / UX Design

Designing responsive, high-performance dashboards using Tailwind CSS and HTMX for rapid client-side transitions without heavy SPA bloat.

6. Cloud Deployment

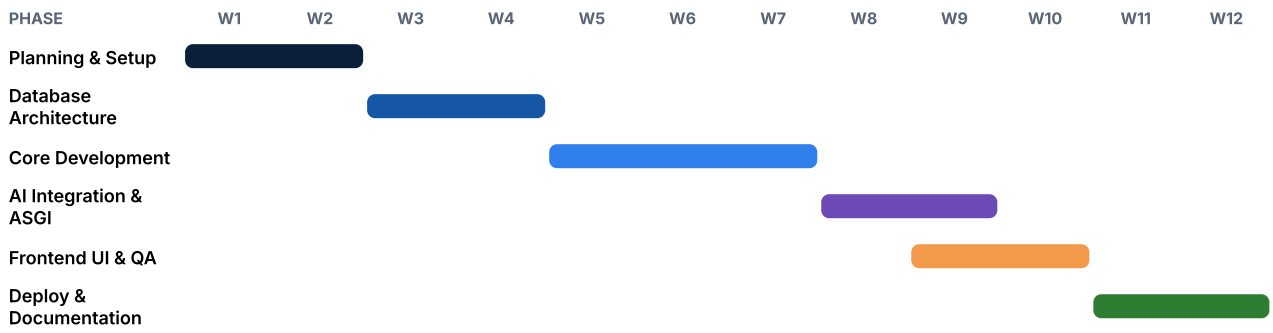
Containerizing the application with Docker, configuring Traefik reverse proxies for wildcard SSL, and deploying via Coolify.

Methodology: Agile / Scrum

The project utilized an iterative Agile methodology. This was critical to accommodate the rapid evolution of external AI diffusion APIs while simultaneously stabilizing the robust e-commerce database schema. Two-week sprints focused on delivering functional architectural slices (e.g., Tenant Routing → Basic Catalog → Asynchronous Generation).

02

GANTT PROJECT TIMELINE (12 WEEKS)



KEY MILESTONES

M1: Schema Finalized

COMPLETED

PostgreSQL relational database schema normalized and initial migrations executed successfully across all app modules.

M2: Tenant Routing

COMPLETED

CustomDomainMiddleware successfully intercepts HTTP Host headers and binds the correct Brand object to the request lifecycle.

M3: AI VTO Working

COMPLETED

End-to-end integration verified: Image upload → Redis → Celery GPU Worker → DB Update → WebSocket Client notification.

M4: Production Ready

COMPLETED

System successfully deployed via Docker and Coolify on an Ubuntu Droplet with Traefik handling wildcard SSL termination.

02

TECHNOLOGY STACK

BACKEND FRAMEWORK

Python / Django 6.1
Provides the robust ORM, routing, and ASGI capability.

RELATIONAL DATABASE

PostgreSQL 14+
Enforces strict tenant isolation and JSONB storage.

ASYNC & BROKER

Celery & Redis
Manages WebSocket pub/sub and GPU task queues.

FRONTEND UI

TailwindCSS & HTMX
Rapid, responsive DOM manipulation without SPA bloat.

INFRASTRUCTURE

Docker & Traefik
Containerization and dynamic reverse proxy routing.

RISK MANAGEMENT

RISK	PROB.	IMPACT	MITIGATION
GPU OOM Crash	HIGH	High	Enforce strict Celery concurrency limits. Route excess tasks to paid APIs.
Cross-Tenant Data Leak	LOW	Crit	Filter ALL querysets explicitly by brand_id bound to the request.
WSS Disconnects	MED	Med	Implement robust client-side JS reconnection logic and fallback polling.
Wildcard SSL Fail	LOW	High	Utilize Traefik DNS-01 Let's Encrypt challenges to automate generation.

TESTING STRATEGY

Unit Testing
Automated Python tests targeting the CustomDomainMiddleware resolution logic and ORM integrity.

Integration Testing
Manual and script-based verification of the E-commerce checkout pipeline.

Stress Testing
Load testing the Daphne ASGI server and Redis broker to ensure WebSocket stability under concurrent connections.

03

UML DIAGRAMS

The Unified Modeling Language (UML) diagrams provide a rigorous structural and behavioral blueprint of the Aura platform. They map the complex multi-tenant logic and asynchronous task flows before code implementation.

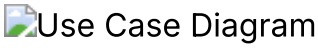


FIGURE 3.1: USE CASE DIAGRAM MAPPING PRIMARY ACTOR CAPABILITIES.

03

CLASS DIAGRAM

FIGURE 3.2: UML CLASS DIAGRAM ILLUSTRATING CORE DOMAIN ENTITIES, ATTRIBUTES, AND MULTIPLICITY.

KEY RELATIONSHIPS

Multi-Tenancy

A single User can own multiple Brand entities. The Brand acts as the structural anchor for all localized e-commerce data within the platform.

Catalog Isolation

Every Product and Order maintains a strict composition relationship with a specific Brand, ensuring rigid data segregation across tenants.

Async AI Tracking

A VTOJob instance bridges a specific User request to a target Product. It holds state strings to monitor background Celery worker progress.

03

SEQUENCE DIAGRAM

Maps the asynchronous Virtual Try-On flow involving WebSockets and external GPU workers.

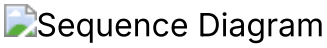


FIGURE 3.3: SEQUENCE DIAGRAM OF THE ASYNCHRONOUS AI VTO FLOW.

ACTIVITY DIAGRAM

Represents the standard user journey from landing on a storefront to finalizing a purchase.

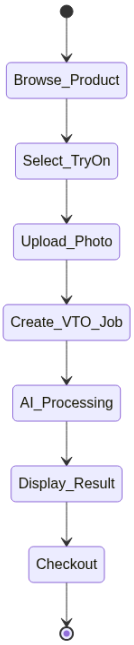


FIGURE 3.4: ACTIVITY DIAGRAM FOR THE E-COMMERCE SHOPPING WORKFLOW.

04

SYSTEM ARCHITECTURE

Aura utilizes a highly scalable, distributed Model-View-Template (MVT) architecture augmented with asynchronous worker nodes (Celery) to handle heavy GPU computation outside the HTTP request-response cycle.

FIGURE 4.1: COMPREHENSIVE SYSTEM ARCHITECTURE DIAGRAM MAPPING SYNCHRONOUS WEB TRAFFIC AGAINST ASYNCHRONOUS BACKGROUND PROCESSING.

04

APPLICATION LAYERS

01. Presentation Layer

Responsibility: Renders UI, handles client inputs, and maintains WebSocket WSS connections.

Components: Tailwind CSS, HTMX, Vanilla JS, Django Templates.

02. Business Logic Layer

Responsibility: Enforces core rules, multi-tenant routing, task delegation, and e-commerce math.

Components: Django Views, CustomDomainMiddleware, Daphne ASGI Server.

03. Data Access Layer

Responsibility: Persists data securely, translates Python objects to SQL queries securely.

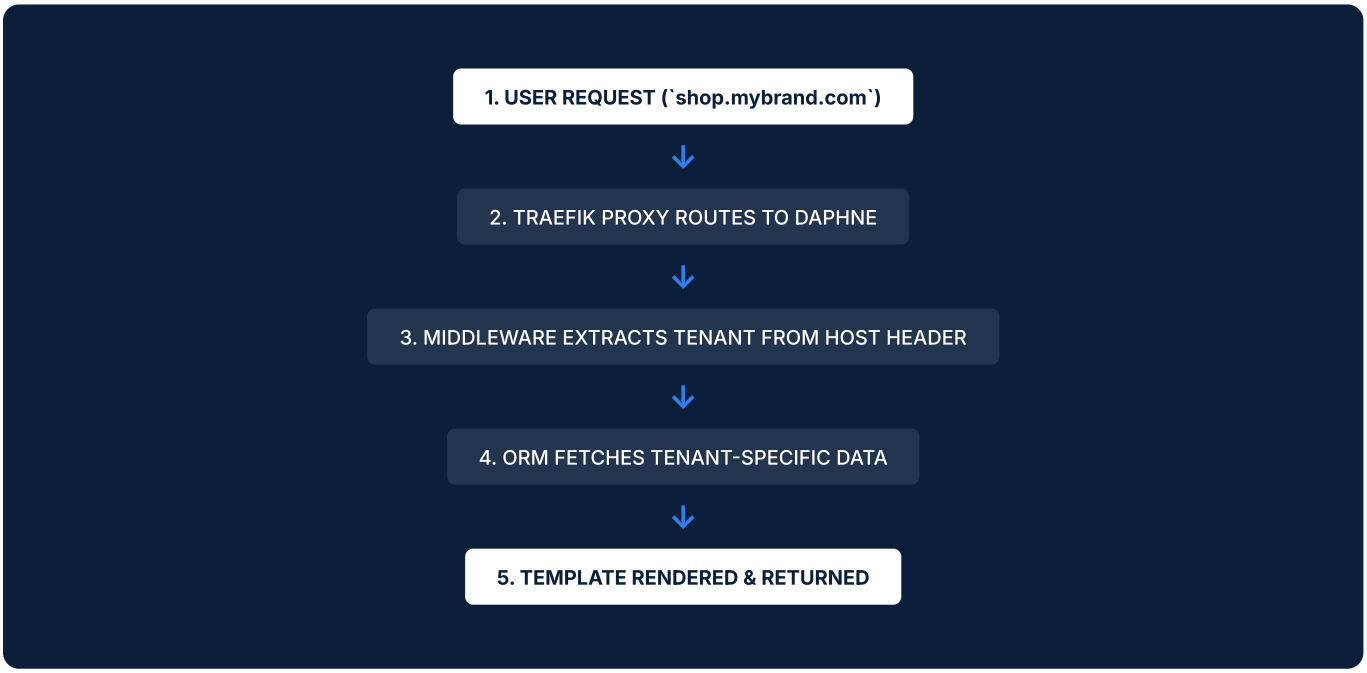
Components: Django ORM, psycopg2, PostgreSQL 14+.

04. Processing Layer

Responsibility: Executes heavy generative AI tasks without blocking the web application.

Components: Redis Broker, Celery Workers, PyTorch / HF Models.

COMPONENT INTERACTION FLOW



05

DATABASE DESIGN

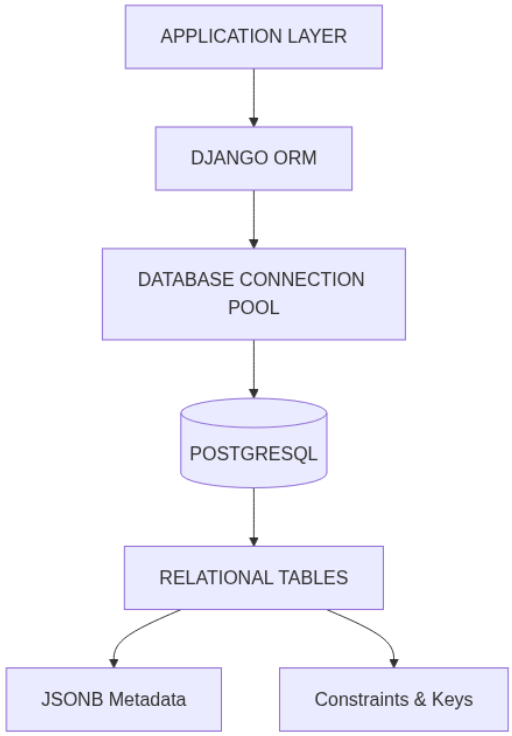


FIGURE 5.1: DATABASE ACCESS AND SCHEMA LAYERING.

DESIGN PRINCIPLES

CONSISTENCY

Strict foreign keys prevent orphaned records.

INTEGRITY

Decimal precision enforced for financials.

SECURITY

brand_id partitioning prevents data leaks.

SCALABILITY

Indexed lookup fields for fast routing.

CORE TABLES

`auth_user`

Central authentication for all human entities (hashed passwords).

`brands_brand`

Core tenant record containing custom_domain routing fields.

`catalog_product`

Store inventory and VTO reference masks.

`orders_order`

E-commerce financial transactions.

`fitting_vtojob`

Tracks asynchronous AI task states.

`fitpassport`

User's saved body preferences.

05

DATABASE SCHEMA STRUCTURE

brands_brand

id	Integer	PK
name	Varchar(200)	
slug	Varchar(100)	UNIQUE
custom_domain	Varchar(255)	NULL
owner_id	Integer	FK

catalog_product

id	Integer	PK
title	Varchar(255)	
price	Decimal(10,2)	
stock	Integer	DEFAULT 0
vto_image	Varchar(100)	NULL
brand_id	Integer	FK

orders_order

id	Integer	PK
total	Decimal(10,2)	
status	Varchar(50)	PENDING
brand_id	Integer	FK
user_id	Integer	FK

fitting_vtojob

id	Integer	PK
status	Varchar(50)	
result_url	Varchar(255)	NULL
product_id	Integer	FK
user_id	Integer	FK

Normalization & Referential Integrity

The schema satisfies **3rd Normal Form (3NF)**. All non-key attributes are strictly dependent on primary keys. Transitive dependencies (like shipping addresses) are abstracted to related profile tables. The database enforces strict ON DELETE CASCADE rules for tenant data; deleting a Brand purges its Catalog to prevent orphaned records.

05

ENTITY-RELATIONSHIP DIAGRAM

FIGURE 5.2: ENTITY-RELATIONSHIP DIAGRAM DETAILING PRIMARY/FOREIGN KEYS AND CARDINALITY.

Cardinality Note: The 1:N relationship between BRAND and nearly all operational tables enforces the strict multi-tenant boundary. The USER table acts as the authentication root, linking securely to both the stores they own and the orders they place.

06

USER MANUAL

CUSTOMER WORKFLOW

1 STOREFRONT → 2 PRODUCT → 3 TRY-ON → 4 GENERATE → 5 CHECKOUT

Page not found (404)

No Brand matches the given query.

Request Method: GET
Request URL: http://localhost:8000/store/owner-brand/
Raised by: apps.brands.views.storefront_view

Using the URLconf defined in `config.urls`, Django tried these URL patterns, in this order:

```
1. admin/ [namespace='admin']
2. django-admin/ [namespace='django_admin']
3. api/
4. p/
5. dashboard/billing/ [name='owner_billing']
6. dashboard/billing/checkout/<int:plan_id>/ [name='create_checkout_session']
7. api/billing/webhook/ [name='stripe_webhook']
8. [name='index']
9. dashboard/ [name='dashboard']
10. dashboard/settings/ [name='brand_settings']
11. dashboard/settings/team/ [name='team_management']
12. store/<slug:slug>/ [name='storefront']
```

The current path, `store/owner-brand/`, matched the last one.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

AI VIRTUAL TRY-ON CAPABILITY

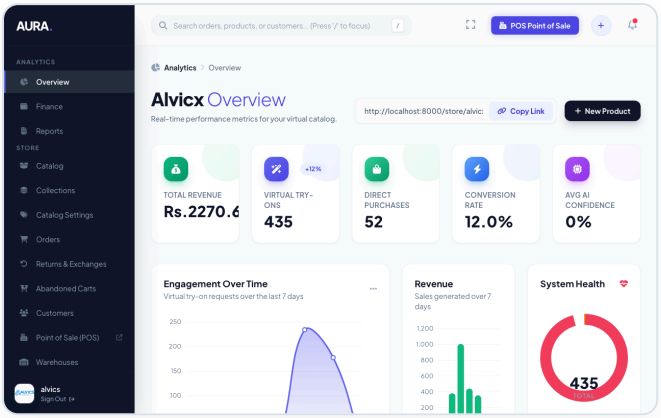
1. USER PHOTO

2. SELECTED PRODUCT

3. AI GENERATED RESULT

06

BRAND OWNER WORKFLOW



1. Authentication

Secure login to isolated tenant admin portal.

2. Analytics Dashboard

Review gross sales, order volume, and VTO metrics.

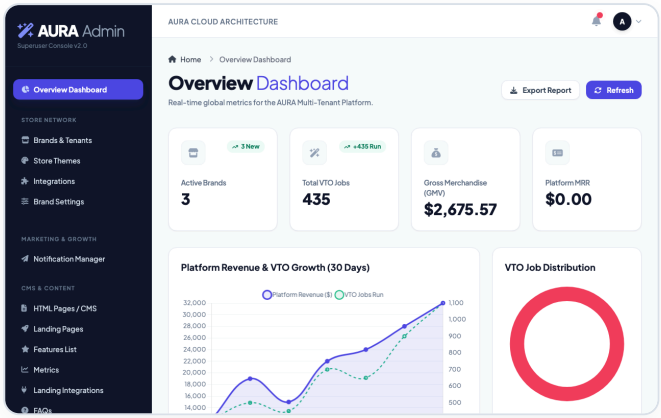
3. Inventory Data Entry

Create products, set pricing, and upload AI masks.

4. Custom Domain

Map external domains dynamically via Settings.

PLATFORM ADMIN WORKFLOW



1. Global Metrics

Monitor platform-wide health and API usage.

2. AI Engine Configuration

Hot-swap between Hugging Face, Replicate, or Local GPU.

3. Tenant Management

Provision, suspend, or upgrade brand subscription tiers.

IMPORTANT DATA ENTRY TIP

Brand Owners must upload high-resolution, front-facing garment images on flat surfaces or ghost mannequins for optimal AI segmentation.

07

INSTALLATION MANUAL

1. SYSTEM → 2. PYTHON → 3. DATABASE → 4. WORKERS

SYSTEM REQUIREMENTS

REQUIREMENT	REC / MIN
OS	Ubuntu 22.04 LTS
RAM	4GB / 16GB Rec.
Python	3.12+
Database	PostgreSQL 14+
Broker	Redis 6.0+

ENVIRONMENT SETUP (.env)

```
DEBUG=False
ALLOWED_HOSTS=*
DATABASE_URL=postgres://user:pass@127.0.0.1:5432/auradb
REDIS_URL=redis://127.0.0.1:6379/0
DJANGO_SECRET_KEY=secure_random_string
```

DEPENDENCIES & SETUP

TERMINAL

```
# 1. Clone repository and set up virtual environment
git clone https://github.com/organization/aura.git
cd aura
python3 -m venv venv
source venv/bin/activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Execute database migrations
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
```

07

RUN APPLICATION (PROCESSES)

WEB SERVER (ASGI)

```
# Start Daphne for WebSockets
daphne -b 0.0.0.0 -p 8000 config.asgi:application
```

AI TASK WORKER

```
# Start Celery queue consumer
celery -A config worker -l info
```

VERIFICATION CHECKLIST

- ✓ PostgreSQL Database Connected
- ✓ Migrations Successfully Executed
- ✓ Daphne ASGI Server Running
- ✓ Celery Worker & Redis Connected
- ✓ Admin Authentication Verified
- ✓ VTO WebSocket Flow Tested

AURA VIRTUAL TRY-ON

ENTERPRISE APPLICATION DEVELOPMENT ASSIGNMENT

— END OF DOCUMENT —